

Load libraries and set user directory paths

```
In [ ]: ## Function to execute shell commands from within R
## Useful when running R in Google Colab or other environments where system calls are needed
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...) # Execute the shell command and capture the output
  cat(paste0(result, collapse = "\n")) # Print the result in a readable format
}

## Function to Load required R packages
## If a package is not installed, it notifies the user
loadPackages = function(pkgs){
  myrequire = function(...){
    suppressWarnings(suppressMessages(suppressPackageStartupMessages(require(...))))
  }
  ok = sapply(pkgs, require, character.only=TRUE, quietly=TRUE) # Check if packages are available
  if (!all(ok)){
    message("There are missing packages: ", paste(pkgs[!ok], collapse=", ")) # Inform user about missing packages
  }
}

## Setup R2U (Ubuntu optimized R package manager) for faster package installation
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_cranapt_jammy.sh",
              "add_cranapt_jammy.sh") # Download installation script
Sys.chmod("add_cranapt_jammy.sh", "0755") # Grant execution permissions to the script
shell_call("./add_cranapt_jammy.sh") # Run the script
bspm::enable() # Enable BSPM (Bridge to System Package Manager) for installing R packages
options(bspm.version.check=FALSE) # Disable BSPM version check
shell_call("rm add_cranapt_jammy.sh") # Remove the installation script after execution

## Install necessary R packages from CRAN
cranPkgs2Install = c("dplyr", "ggpubr", "Seurat", "cowplot",
                    "Rtsne", "hdf5r", "patchwork")
install.packages(cranPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

```
In [1]: ## To simplify package loading, we created the loadPackages() function
## If you don't have this function, you should load packages using library(name_of_package)
pkgs = c("Seurat", "dplyr", "patchwork", "ggplot2")
loadPackages(pkgs) # Load the specified packages

# Important
# Update scw01 directory to match your personal user directory
# To match your personal user directory. This is where you read and write data
mydir <- "/content"
```

```
Attaching package: 'SeuratObject'
```

```
The following objects are masked from 'package:base':
```

```
intersect, t
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

Multi-modal Data Analysis

In this notebook, we will embark on the exciting journey of analyzing multi-modal data, specifically focusing on the RNA and protein expression levels in individual cells. We will be guided by a comprehensive tutorial from Seurat, which you can find in detail [here](#).

The dataset we will be working with consists of 8,617 cord blood mononuclear cells (CBMCs), the original article is [Simultaneous epitope and transcriptome measurement in single cells](#).

These cells have been meticulously analyzed to provide both transcriptomic measurements and abundance estimates for 11 surface proteins. The protein levels have been quantified using DNA-barcoded antibodies, allowing for precise and reliable data.

To begin our analysis, we will load two count matrices: one containing the RNA measurements and the other containing the antibody-derived tags (ADT). This dual dataset will enable us to explore and understand the intricate relationships between RNA expression and protein levels in these individual cells.

Here's a brief overview of our dataset as described by Seurat: > "In this analysis, we examine a dataset of 8,617 cord blood mononuclear cells (CBMCs), where transcriptomic measurements are paired with abundance estimates for 11 surface proteins, quantified using DNA-barcoded antibodies. Initially, we load two count matrices: one for the RNA measurements and one for the antibody-derived tags (ADT)."

By following this tutorial, we aim to gain insights into the biological mechanisms at play and the interactions between RNA and protein expressions at a single-cell level.

Get ready for a deep dive into multi-modal data analysis!

```
In [10]: # Download raw single-cell RNA-seq and ADT (antibody-derived tag) data from the NCBI GEO datab
shell_call("wget ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE100nnn/GSE100866/suppl/GSE100866_CBM
shell_call("wget ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE100nnn/GSE100866/suppl/GSE100866_CBM
```

```
In [12]: # Load in the RNA UMI matrix
# Note: The dataset also contains ~5% of mouse cells, which we can use as negative controls fo
# For this reason, the gene expression matrix has HUMAN_ or MOUSE_ appended to the beginning o

cbmc.rna <- as.sparse(read.csv(file = "GSE100866_CBMC_8K_13AB_10X-RNA_umi.csv.gz", sep = ",",

# read.csv: reads a CSV file into a data frame, file argument specifies the path to the CSV fi
# and (paste0) concatenates the directory stored in mydir with the filename.
# sep = ",": Indicates that the delimiter used in the CSV file is a comma.
# header = TRUE: Specifies that the first row of the CSV file contains column names.
# row.names = 1: Specifies that the first column of the CSV file should be used as row names
# as.sparse(): This function converts the data frame into a sparse matrix.
# Sparse matrices are used to efficiently store data with a lot of zero values, which is commo

# To make life a bit easier going forward, we're going to discard all but the top 100 most hig
# and remove the 'HUMAN_' from the CITE-seq prefix

cbmc.rna <- CollapseSpeciesExpressionMatrix(cbmc.rna)

# This command processes the cbmc.rna matrix to handle or collapse gene expression data from p
# ensuring that the matrix is properly formatted for further analysis.

# Load the ADT UMI matrix (for protein-level measurements)
cbmc.adt <- as.sparse(read.csv(file = "GSE100866_CBMC_8K_13AB_10X-ADT_umi.csv.gz", sep = ",",

# Note: Ensure that both RNA and ADT matrices have identical column names (i.e., the same cell
# This command used to compare the column names of two data frames

all.equal(colnames(cbmc.rna), colnames(cbmc.adt)) # Should return TRUE
```

TRUE

```
In [13]: # Create a Seurat object for the scRNA-seq data
cbmc <- CreateSeuratObject(counts = cbmc.rna)

# Verify the available assays (default is RNA).
# The cbmc object contains an assay storing RNA measurement
Assays(cbmc)

# Create a new assay to store ADT information
adt_assay <- CreateAssayObject(counts = cbmc.adt)

# Add the ADT assay to the Seurat object
cbmc[["ADT"]] <- adt_assay

# Verify that the Seurat object now contains both RNA and ADT assays
Assays(cbmc)

# Extract a List of features (antibodies) measured in the ADT assay
rownames(cbmc[["ADT"]])

# Note: We can easily switch back and forth between the two assays to specify the default for
```

```
# List the current default assay (should be RNA)
DefaultAssay(cbmc)

# Switch the default assay to ADT
DefaultAssay(cbmc) <- "ADT"
DefaultAssay(cbmc) # Should now return "ADT"
```

Warning message:

“Feature names cannot have underscores ('_'), replacing with dashes ('-')”

'RNA'

'RNA' · 'ADT'

'CD3' · 'CD4' · 'CD8' · 'CD45RA' · 'CD56' · 'CD16' · 'CD10' · 'CD11c' · 'CD14' · 'CD19' · 'CD34' · 'CCR5' · 'CCR7'

'RNA'

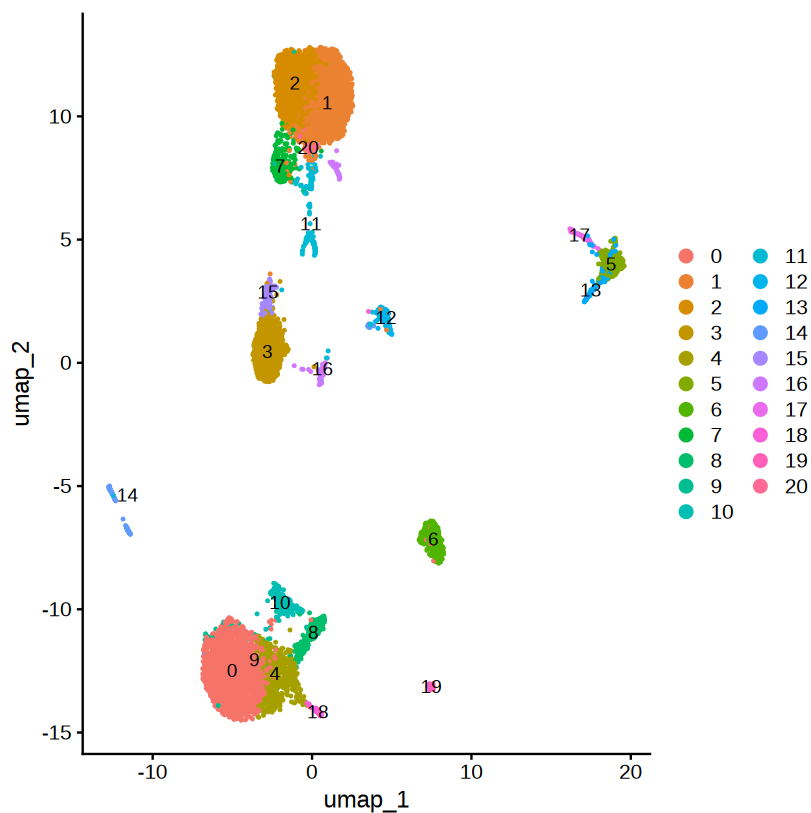
'ADT'

Clustering

```
In [ ]: # Note that all operations below are performed on the RNA assay Set and verify that the
# Default assay is RNA
DefaultAssay(cbmc) <- "RNA" # This command sets the default assay for the cbmc object to be th
DefaultAssay(cbmc) # Show the Assay

# Perform visualization and clustering steps
cbmc <- NormalizeData(cbmc) # This command normalizes the gene expression data in the cbmc obj
cbmc <- FindVariableFeatures(cbmc) # This command identifies most variable features in the dat
cbmc <- ScaleData(cbmc) # This command scales the data, adjusting the mean of gene expression
cbmc <- RunPCA(cbmc, verbose = FALSE) # This command performs a Principal Component Analysis (
cbmc <- FindNeighbors(cbmc, dims = 1:30) # This command finds the nearest neighbors of each ce
cbmc <- FindClusters(cbmc, resolution = 0.8) # This command performs clustering of the cells u
cbmc <- RunUMAP(cbmc, dims = 1:30) # This command performs UMAP (Uniform Manifold Approximatio
```

```
In [15]: DimPlot(cbmc, label = TRUE) # This command generates a scatter plot of the cells in UMAP space
```



Visualize multiple modalities side-by-side

Now that we have successfully clustered our scRNA-seq profiles, we can move on to visualizing the expression of either protein or RNA molecules in our dataset. Seurat offers several methods to switch between different modalities and specify which modality you are interested in analyzing or visualizing.

This is especially important because, in some cases, the same feature can be present in multiple modalities. For instance, in our dataset, we have independent measurements of the B cell marker CD19, both at the protein level and RNA level. Being able to switch between these modalities allows us to comprehensively explore and understand the biological significance of these features.

By leveraging Seurat's capabilities, we can gain deeper insights into the relationships and interactions between RNA and protein expressions within individual cells, ultimately enhancing our understanding of the underlying biological processes.

If you have any questions or need further assistance with the analysis, feel free to ask!

```
In [17]: # Normalize ADT data
DefaultAssay(cbm) <- "ADT" # This command sets the default assay for the cbm object to be the ADT assay
cbm <- NormalizeData(cbm, normalization.method = "CLR", margin = 2)
# This command normalizes the ADT data in the cbm object using the CLR (Centered Log Ratio) method
# The margin = 2 parameter indicates that normalization will be applied to the columns (genes/proteins)

DefaultAssay(cbm) <- "RNA" # This command resets the default assay for the cbm object to be the RNA assay

# Note that the following command is an alternative but returns the same result
cbm <- NormalizeData(cbm, normalization.method = "CLR", margin = 2, assay = "ADT")
```

```

# Now, we will visualize CD14 Levels for RNA and protein.
# By setting the default assay, we can visualize one or the other
DefaultAssay(cbm) <- "ADT"
p1 <- FeaturePlot(cbm, "CD19", cols = c("lightgrey", "darkgreen")) + # This command creates a
  ggtitle("CD19 protein") # Add a title to the plot p1

DefaultAssay(cbm) <- "RNA" # This command resets the default assay to RNA.
p2 <- FeaturePlot(cbm, "CD19") + ggtitle("CD19 RNA") # This command creates a FeaturePlot of

# Place plots side-by-side
p1 | p2 # Show p1 and p2 side by side

# Alternately, we can use specific assay keys to specify a specific modality Identify the key
Key(cbm[["RNA"]])
Key(cbm[["ADT"]])

# These commands identify the keys for the RNA and ADT assays, respectively. Keys are prefixes
# Now, we can include the key in the feature name, which overrides the default assay
p1 <- FeaturePlot(cbm, "adt_CD19", cols = c("lightgrey", "darkgreen")) + ggtitle("CD19 protei
p2 <- FeaturePlot(cbm, "rna_CD19") + ggtitle("CD19 RNA")
p1 | p2

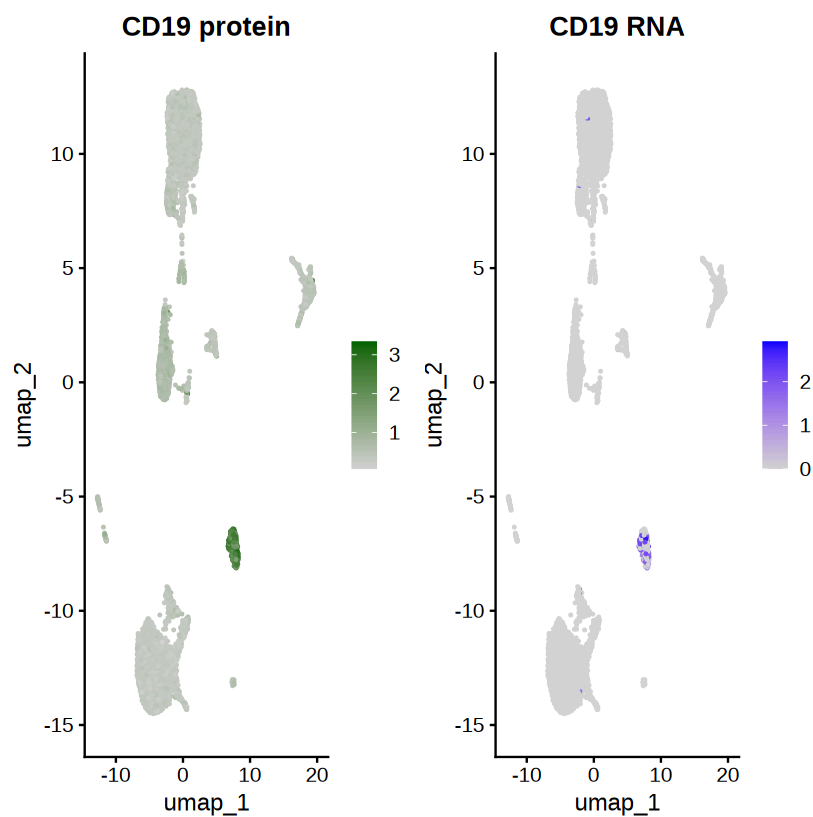
```

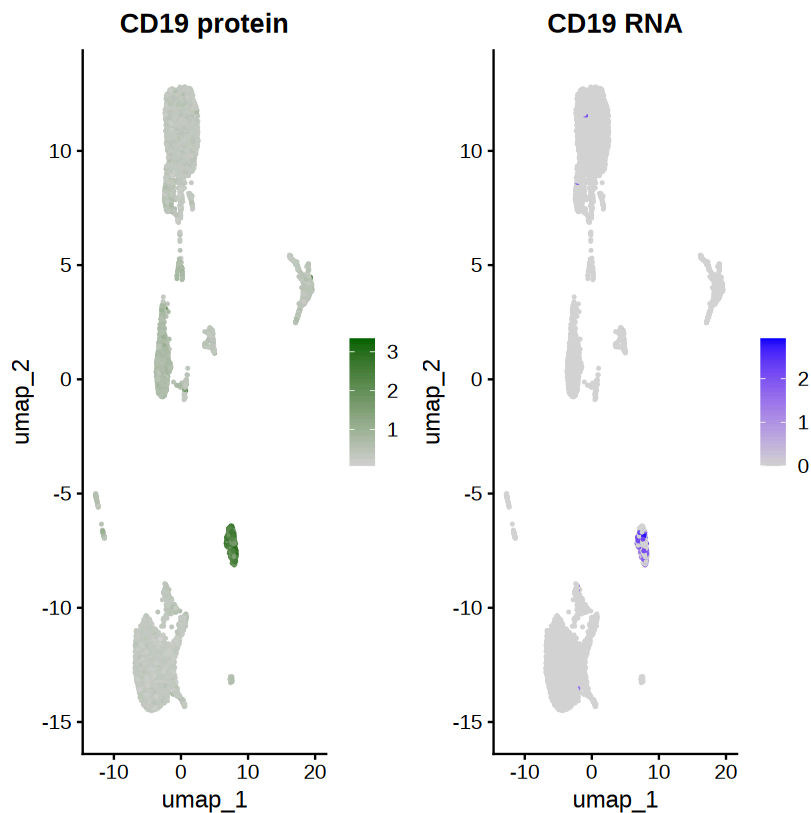
Normalizing across cells

Normalizing across cells

'rna_'

'adt_'





Identify cell surface markers for scRNA-seq clusters

```
In [18]: # As we know, CD19 is a B cell marker, we can identify cluster 6 as expressing CD19 on the surface
VlnPlot(cbm, "adt_CD19") # Create a violin plot

# We can also identify alternative protein and RNA markers for this cluster through differential expression
adt_markers <- FindMarkers(cbm, ident.1 = 6, assay = "ADT") # This function identifies differentially expressed markers
rna_markers <- FindMarkers(cbm, ident.1 = 6, assay = "RNA") # This function identifies differentially expressed markers
# ident.1 = 6: Specifies that cluster 6 is the group of interest for which markers are being identified
# assay: Indicates that the ADT/RNA assay data should be used for the analysis.
# adt/rna_markers: Stores the results of the marker identification for the ADT/RNA assay.

# Display the top markers
head(adt_markers)
head(rna_markers)
```

Additional visualizations of multimodal data

```
In [19]: # Draw ADT scatter plots (like biaxial plots for FACS).
# Note that you can even 'gate' cells if desired by using HoverLocator and FeatureLocator
FeatureScatter(cbm, feature1 = "adt_CD19", feature2 = "adt_CD3")

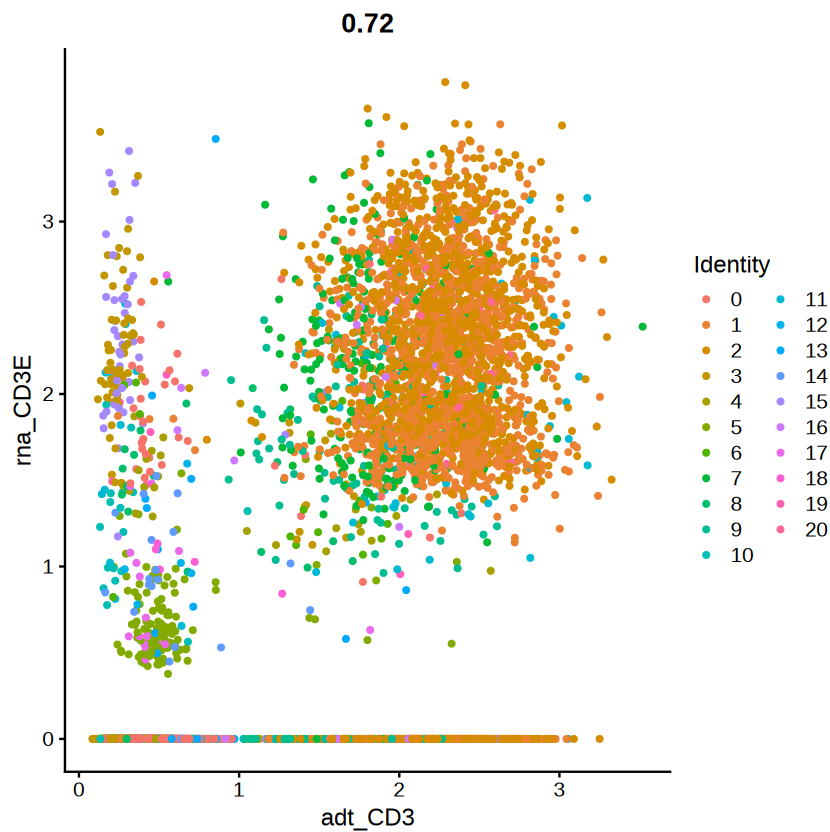
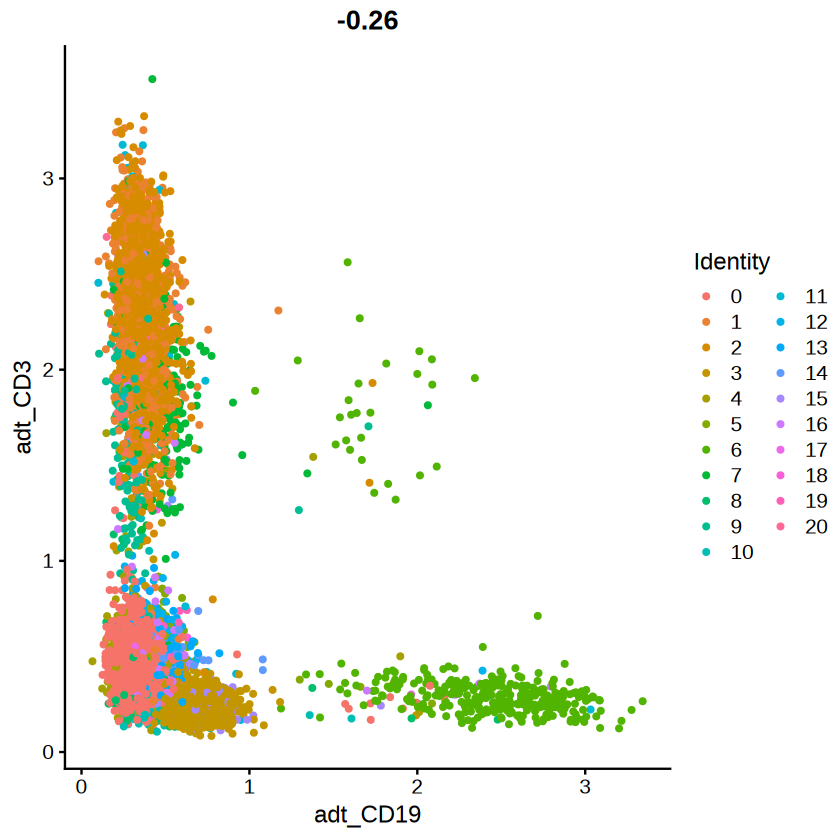
# View relationship between protein and RNA
FeatureScatter(cbm, feature1 = "adt_CD3", feature2 = "rna_CD3E")

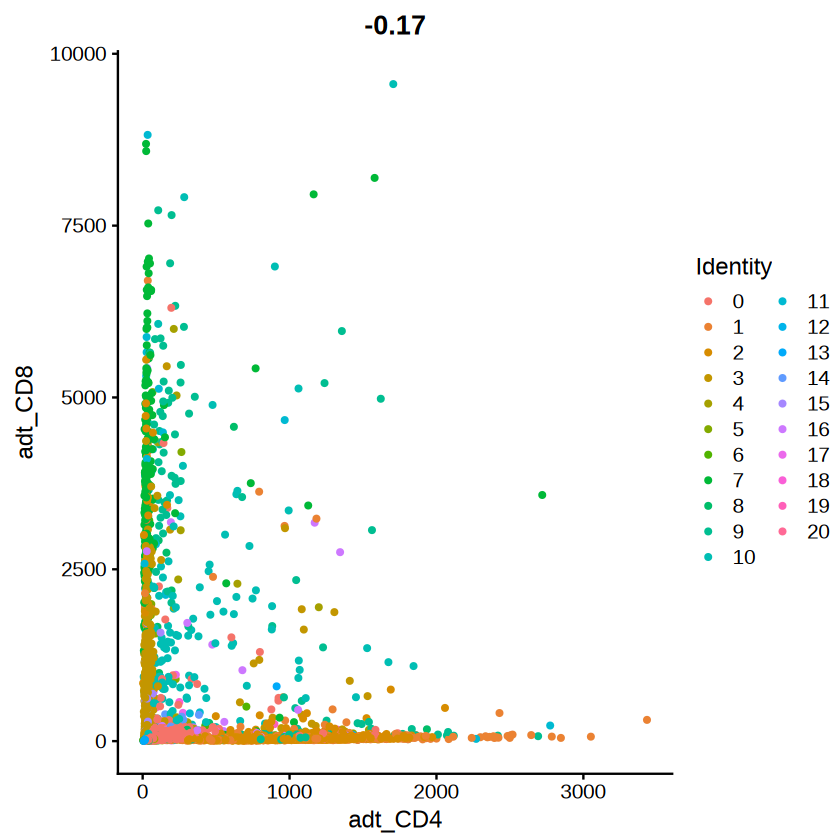
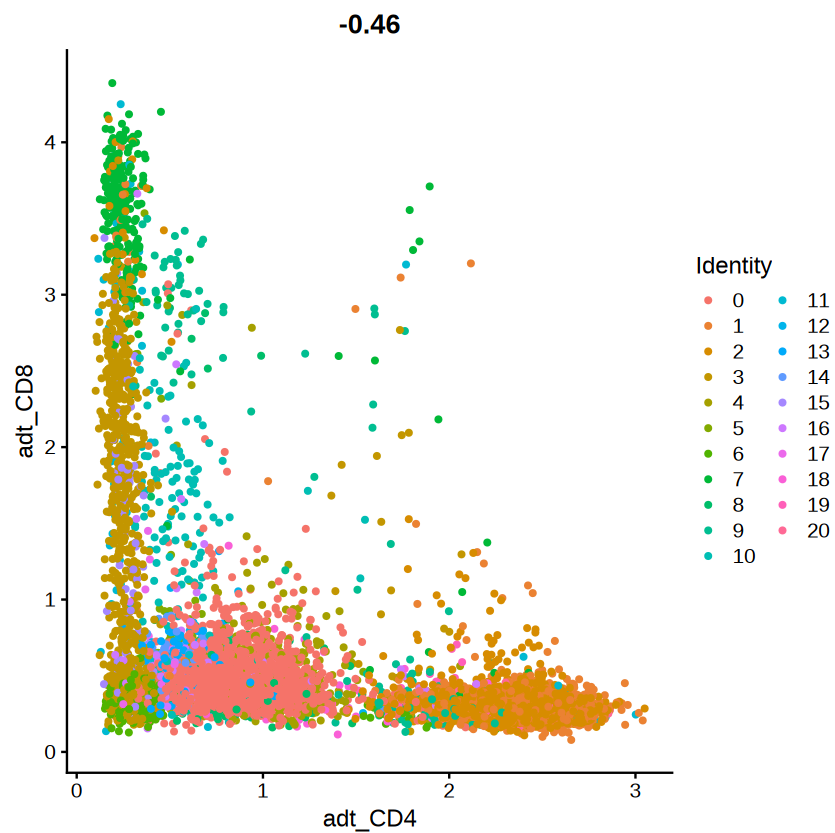
FeatureScatter(cbm, feature1 = "adt_CD4", feature2 = "adt_CD8")

# Let's look at the raw (non-normalized) ADT counts. You can see the values are quite high,
```

```
# particularly in comparison to RNA values. This is due to the significantly higher protein  
# copy number in cells, which significantly reduces 'drop-out' in ADT data
```

```
FeatureScatter(cbm, feature1 = "adt_CD4", feature2 = "adt_CD8", slot = "counts")  
# slot = "counts": This parameter specifies that the raw (non-normalized) counts from the ADT
```





Loading data from 10X multi-modal experiments

NOTE: DO NOT RUN THIS SECTION

We are including it for you reference if you will need to process multi-modal 10x data from the raw data.

```
In [ ]: # Load the 10X Genomics PBMC dataset from the specified directory
# This data includes gene expression and antibody capture information
pbmc10k.data <- Read10X(data.dir = "../data/pbmc10k/filtered_feature_bc_matrix/")

# Clean up the row names of the Antibody Capture data to remove unwanted suffixes,
# specifically the '_TotalSeqB' and any 'control' prefixes
rownames(x = pbmc10k.data[["Antibody Capture"]]) <- gsub(pattern = "_[control_]*TotalSeqB", re
  x = rownames(x = pbmc10k.data[["Antibody Capture"]]))
# rownames(): This function retrieves or sets the row names of a matrix or data frame.
# gsub(): This function performs replacement of patterns in strings. Here, it is used to
# Start with an underscore followed by "control" (optionally repeated) and end with "TotalSeqB"
# replacement = "": This indicates that the matched pattern should be replaced with an empty s
# x = rownames(x = pbmc10k.data[["Antibody Capture"]]): This specifies the input strings (row

# Create a Seurat object for the gene expression data,
# filtering to keep genes detected in at least 3 cells and cells with at least 200 features
pbmc10k <- CreateSeuratObject(counts = pbmc10k.data[["Gene Expression"]], min.cells = 3, min.f

# Normalize the gene expression data using Log normalization
pbmc10k <- NormalizeData(pbmc10k)

# Add the ADT data (Antibody Derived Tags) to the Seurat object as a separate assay
# This ensures we can analyze protein expression alongside RNA data
pbmc10k[["ADT"]] <- CreateAssayObject(pbmc10k.data[["Antibody Capture"]][, colnames(x = pbmc10

# Normalize the ADT data using CLR (Centered Log Ratio) normalization method
# This is important for correcting technical variation in protein expression data
pbmc10k <- NormalizeData(pbmc10k)

# Add the ADT data (Antibody Derived Tags) to the Seurat object as a separate assay
# This ensures we can analyze protein expression alongside RNA data
pbmc10k[["ADT"]] <- CreateAssayObject(pbmc10k.data[["Antibody Capture"]][, colnames(x = pbmc10

# Normalize the ADT data using CLR (Centered Log Ratio) normalization method
# This is important for correcting technical variation in protein expression data
pbmc10k <- NormalizeData(pbmc10k, assay = "ADT", normalization.method = "CLR")

# Create scatter plots to visualize the relationship between different features:
# Plot 1: CD19 vs. CD3 protein expression
plot1 <- FeatureScatter(pbmc10k, feature1 = "adt_CD19", feature2 = "adt_CD3", pt.size = 1)

# Plot 2: CD4 vs. CD8a protein expression
plot2 <- FeatureScatter(pbmc10k, feature1 = "adt_CD4", feature2 = "adt_CD8a", pt.size = 1)

# Plot 3: CD3 protein expression vs. CD3E RNA expression
plot3 <- FeatureScatter(pbmc10k, feature1 = "adt_CD3", feature2 = "CD3E", pt.size = 1)
```

```
# Combine the three scatter plots into one figure and remove the legend for clarity  
(plot1 + plot2 + plot3) & NoLegend()
```